```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.kernel_approximation import RBFSampler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files Wine dataset.csv

**Wine dataset.csv**(text/csv) - 11482 bytes, last modified: 8/17/2026 - 100% done

Saving Wine dataset.csv to Wine dataset (1).csv

```
df = pd.read_csv("Wine dataset.csv")

df.columns = df.columns.str.strip()

print("Dataset Shape:", df.shape)

display(df.head())
```

Dataset Shape: (178, 14)

| | class | Alcohol | Malic acid | Ash | Alcalinity of ash | Magnesium | Total phenols | Flavanoids | Nonflavanoid phenols | Proanthocyanins | Color intensity | Hue | OD280/OD315 of diluted wines | Proline |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 1 | 14.23 | 1.71 | 2.43 | 15.6 | 127 | 2.80 | 3.06 | 0.28 | 2.29 | 5.64 | 1.04 | 3.92 | 1065 |
| **1** | 1 | 13.20 | 1.78 | 2.14 | 11.2 | 100 | 2.65 | 2.76 | 0.26 | 1.28 | 4.38 | 1.05 | 3.40 | 1050 |
| **2** | 1 | 13.16 | 2.36 | 2.67 | 18.6 | 101 | 2.80 | 3.24 | 0.30 | 2.81 | 5.68 | 1.03 | 3.17 | 1185 |
| **3** | 1 | 14.37 | 1.95 | 2.50 | 16.8 | 113 | 3.85 | 3.49 | 0.24 | 2.18 | 7.80 | 0.86 | 3.45 | 1480 |
| **4** | 1 | 13.24 | 2.59 | 2.87 | 21.0 | 118 | 2.80 | 2.69 | 0.39 | 1.82 | 4.32 | 1.04 | 2.93 | 735 |

```
X = df.drop(
    columns=["class"]
)
```

```
y = df["class"]

print("Features:", X.shape[1])
print("Classes:")
print(y.value_counts())
```

```
Features: 13
Classes:
class
2    71
1    59
3    48
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 124
Testing samples: 54
```

```
rbf_model = make_pipeline(

    RBFSampler(
        gamma=0.05,
        n_components=200,
        random_state=42
    ),

    LogisticRegression(
        max_iter=2000
    )
)

rbf_model.fit(
    X_train,
    y_train
)

rbf_predictions = rbf_model.predict(
    X_test
)

rbf_accuracy = accuracy_score(
```

```
    y_test,
    rbf_predictions
)

print(
    "RBF Accuracy:",
    rbf_accuracy
)
```

```
RBF Accuracy: 1.0
```

```
print("Model Comparison")
print("--------------------------")

# Define, train, and evaluate the KNN model
knn_model = KNeighborsClassifier(n_neighbors=5) # Using a default n_neighbors, can be adjusted
knn_model.fit(X_train, y_train)
knn_predictions = knn_model.predict(X_test)
knn_accuracy = accuracy_score(y_test, knn_predictions)

print(
    "KNN Accuracy:",
    round(knn_accuracy * 100, 2),
    "%"
)

print(
    "RBF Accuracy:",
    round(rbf_accuracy * 100, 2),
    "%"
)
```

```
Model Comparison
--------------------------
KNN Accuracy: 94.44 %
RBF Accuracy: 100.0 %
```

```
fig, axes = plt.subplots(
    1, 2,
    figsize=(12, 5)
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    knn_predictions,
    ax=axes[0]
)

axes[0].set_title(
    "KNN Confusion Matrix"
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    rbf_predictions,
    ax=axes[1]
```

```
)

axes[1].set_title(
    "RBF Confusion Matrix"
)

plt.tight_layout()
plt.show()
```